

Security Audit

Report for XAUM

Contracts

Date: March 21, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	1
1.3.1 Software Security	2
1.3.2 DeFi Security	2
1.3.3 NFT Security	2
1.3.4 Additional Recommendation	2
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Software Security	5
2.1.1 The <code>closeOrderByOperator()</code> function invocation may fail	5
2.1.2 Potential reentrancy attack	6
2.1.3 Potential DoS in function <code>_removeUserOrder()</code>	7
2.1.4 Potential DoS or unfairness due to incorrect calculation of allowanceAmount	8
2.2 DeFi Security	10
2.2.1 The pending XAUM may be lost	10
2.2.2 Incorrect round down in function <code>_burnShareAfterSendDollar()</code>	11
2.2.3 Potential inflation attack in function <code>createOrder()</code>	12
2.2.4 Lack of revoking approvals	14
2.2.5 Lack of check on <code>minDollarPriceAllowed</code> variable	15
2.2.6 Potential overflow during type casting	16
2.2.7 Lack of sanity checks in function <code>setDCA()</code>	18
2.2.8 Improper order completion logic	18
2.2.9 The <code>fixedPrice</code> for all the stable tokens is the same	19
2.3 Additional Recommendation	20
2.3.1 Lack of invoking function <code>_disableInitializers()</code>	20
2.3.2 Redundant rounding up in total fee calculation	20
2.3.3 Redundant <code>XAUMDCAForTest</code>	20
2.4 Note	21
2.4.1 Ensure enough XAUM tokens to pay users in <code>XAUMDCAMinter</code> contract	21
2.4.2 Potential centralization risk	21
2.4.3 Excess allowance can reduce transaction count	21
2.4.4 Not checking the <code>data</code> parameter of the <code>_executeOrder()</code> function	22

Report Manifest

Item	Description
Client	MatrixPort
Target	XAUM Contracts

Version History

Version	Date	Description
1.0	March 21, 2025	First release

Signature



Digitally signed by: BlockSec (<https://blocksec.com>)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2025-03-21 04:04:50+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository of XAUM Contracts¹ of MatrixPort. Note that we only focus on the [XAUMDCA](#), [XAUMDCAMinter](#), and [XAUMDCARouter](#) contracts. Other contracts and source code files in the repository are out of scope for this audit. The auditing process is iterative. Specifically, we will audit the commits that fix the discovered issues. If there are new issues, we will continue this process.

Project	Version	Commit Hash
XAUM Contracts	Version 1	90de984ee4aad006dde4a61d02e1acee02e05d94
	Version 2	c4155b048f92057a9d6128c9c1756fb46b221e97

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.

¹<https://github.com/Matrixdock-RWA/RWA-Contracts>

- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.
We show the main concrete checkpoints in the following.

1.3.1 Software Security

- * Reentrancy
- * DoS
- * Access control
- * Data handling and data flow
- * Exception handling
- * Untrusted external call and control flow
- * Initialization consistency
- * Events operation
- * Error-prone randomness
- * Improper use of the proxy system

1.3.2 DeFi Security

- * Semantic consistency
- * Functionality consistency
- * Permission management
- * Business logic
- * Token operation
- * Emergency mechanism
- * Oracle security
- * Whitelist and blacklist
- * Economic impact
- * Batch transfer

1.3.3 NFT Security

- * Duplicated item
- * Verification of the token receiver
- * Off-chain metadata security

1.3.4 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology² and Common Weakness Enumeration³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following four categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **thirteen** potential security issues. Besides, we have **three** recommendations and **four** notes.

- Medium Risk: 5
- Low Risk: 8
- Recommendation: 3
- Note: 4

ID	Severity	Description	Category	Status
1	Medium	The <code>closeOrderByOperator()</code> function invocation may fail	Software Security	Fixed
2	Medium	Potential reentrancy attack	Software Security	Fixed
3	Low	Potential DoS in function <code>_removeUserOrder()</code>	Software Security	Confirmed
4	Low	Potential DoS or unfairness due to incorrect calculation of allowanceAmount	Software Security	Confirmed
5	Medium	The pending XAUM may be lost	DeFi Security	Fixed
6	Medium	Incorrect round down in function <code>_burnShareAfterSendDollar()</code>	DeFi Security	Fixed
7	Medium	Potential inflation attack in function <code>createOrder()</code>	DeFi Security	Fixed
8	Low	Lack of revoking approvals	DeFi Security	Fixed
9	Low	Lack of check on <code>minDollarPriceAllowed</code> variable	DeFi Security	Fixed
10	Low	Potential overflow during type casting	DeFi Security	Fixed
11	Low	Lack of sanity checks in function <code>setDCA()</code>	DeFi Security	Fixed
12	Low	Improper order completion logic	DeFi Security	Confirmed
13	Low	The <code>fixedPrice</code> for all the stable tokens is the same	DeFi Security	Fixed
14	-	Lack of invoking function <code>_disableInitializers()</code>	Recommendation	Fixed
15	-	Redundant rounding up in total fee calculation	Recommendation	Fixed
16	-	Redundant <code>XAUMDCAForTest</code>	Recommendation	Fixed
17	-	Ensure enough XAUM tokens to pay users in <code>XAUMDCAMinter</code> contract	Note	-
18	-	Potential centralization risk	Note	-
19	-	Excess allowance can reduce transaction count	Note	-

20	-	Not checking the <code>data</code> parameter of the <code>_executeOrder()</code> function	Note	-
----	---	---	------	---

The details are provided in the following sections.

2.1 Software Security

2.1.1 The `closeOrderByOperator()` function invocation may fail

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The operator role has the authority to close the orders of blacklisted users. Blacklisted users whose `order.status != STATUS_ACTIVE` will not be closed. Their status may be `STATUS_COMPLETED_WITHOUT_CLAIM`. In this case, the `closeOrderByOperator()` function will fail.

```

454 function closeOrderByOperator(uint64 id) public onlyOperator {
455     Order storage order = orders[id];
456     require(userBlacklist[order.owner] == true, 'DCA_NOT_IN_BLACKLIST');
457     _closeOrder(order, order.owner, id, legalAccount);
458 }
```

Listing 2.1: XAUMCDA.sol

```

460 function _closeOrder(Order storage order, address user, uint64 id, address receiver) internal
461 {
462     require(order.status == STATUS_ACTIVE, 'DCA_INVALID_ORDER_STATUS');
463     uint xaumBalance = order.xaumBalance;
464     uint dollarBalance = order.dollarBalance;
465
466     if (xaumBalance > 0) {
467         order.xaumBalance = 0;
468         IERC20(xaum).safeTransfer(receiver, xaumBalance);
469     }
470     if (dollarBalance > 0) {
471         order.dollarBalance = 0;
472         if (isRebaseToken) {
473             uint256 shareBalance = order.dollarShareBalance;
474             order.dollarShareBalance = 0;
475             dollarBalance = getAmountByShare(shareBalance);
476             totalShares -= shareBalance;
477         }
478         IERC20(dollar).safeTransfer(receiver, dollarBalance);
479     }
480     order.status = STATUS_CANCELED;
481     _removeUserOrder(user, id);
482     emit CloseOrder(user, id, receiver, xaumBalance, dollarBalance);
```

483 }

Listing 2.2: XAUMCDA.sol

Impact When the blacklisted user's order.status is `STATUS_COMPLETED_WITHOUT_CLAIM`, the `closeOrderByOperator()` function will fail to call.

Suggestion Revise the code logic accordingly.

2.1.2 Potential reentrancy attack

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The `_swapDollarToStableToken()` function swaps dollar tokens to stable tokens via a low-level call to the `adapter`. Although there is a whitelist check on the `adapter`, the `adapter` itself may call other untrusted contracts during execution. Consequently, malicious actors may take over the call flow to reenter the contract. For example, the swap operation will first transfer the dollar to the adapter which will decrease the dollar balance. After the swap, the shares will be burned. If the swap process has an external call, the malicious user can re-enter the function `createOrder()` to mint shares with a relatively low amount of dollars.

```

345 function _swapDollarToStableToken(Order storage order, address adapter, bytes memory data)
    internal returns (uint dollarDelta, uint stableTokenDelta){
346     // Cache storage variables
347     uint256 dollarBalance = order.dollarBalance;
348     uint256 dollarPerTrade = order.dollarPerTrade;
349     address dollarAddr = dollar;
350     address stableTokenAddr = stableToken;
351
352
353     // Get balances once
354     uint256 dollarBalanceBefore = IERC20(dollarAddr).balanceOf(address(this));
355     uint256 stableTokenBalanceBefore = IERC20(stableTokenAddr).balanceOf(address(this));
356
357
358     // Calculate allowance amount
359     uint256 allowanceAmount;
360     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
361         allowanceAmount = getAmountByShare(order.dollarShareBalance) + 1; // for share amount
            convert calculate precision
362     } else {
363         allowanceAmount = dollarPerTrade;
364     }
365     IERC20(dollarAddr).forceApprove(adapter, allowanceAmount * 11 / 10); // approve more
366
367
368     // Execute swap
369     (bool success, bytes memory result) = adapter.call(data);
370     if (!success) {
371         revert CallFailed(result);

```

```
372     }
373
374
375     // Get balances after swap
376     uint256 dollarBalanceAfter = IERC20(dollarAddr).balanceOf(address(this));
377     uint256 stableTokenBalanceAfter = IERC20(stableTokenAddr).balanceOf(address(this));
378
379
380     // Calculate deltas
381     dollarDelta = dollarBalanceBefore - dollarBalanceAfter;
382     stableTokenDelta = stableTokenBalanceAfter - stableTokenBalanceBefore;
383
384
385     // Validate deltas
386     require(dollarDelta > 0, 'DCA_DOLLAR_DELTA_MUST_POSITIVE');
387     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
388         require(dollarDelta <= allowanceAmount, 'DCA_INVALID_DOLLAR_DELTA');
389     } else {
390         require(dollarDelta <= dollarPerTrade * 11 / 10 && dollarDelta <= dollarBalance, '
            DCA_INVALID_DOLLAR_DELTA'); // allow trade a little more than dollarPerTrade every
            trade
391     }
392     require(stableTokenDelta > 0, 'DCA_INVALID_STABLE_TOKEN_AMOUNT');
393
394
395     // Validate price
396     uint price = uint224(uint224(stableTokenDelta) * (2**112) / uint112(dollarDelta));
397     require(price >= minDollarPriceAllowed, 'DCA_INVALID_DOLLAR_PRICE');
398
399
400     return (dollarDelta, stableTokenDelta);
401 }
```

Listing 2.3: XAUMCDA.sol

Impact If the swap process has an external call, the malicious user can re-enter the function `createOrder()` to mint shares with a relatively low amount of dollars.

Suggestion Apply the ReentrancyGuard on the external functions in the contract `XAUMCDA`.

2.1.3 Potential DoS in function `_removeUserOrder()`

Severity Low

Status Confirmed

Introduced by Version 1

Description When removing a user's order, `_removeUserOrder()` will traverse all of the user's orders. Since users can create orders without limitation on the length of created orders, it is possible that the array will be too long, resulting in revert due to insufficient gas.

```
504     function _removeUserOrder(address user, uint64 id) internal {
505         uint64[] storage orderIds = userOrders[user];
506         for (uint i = 0; i < orderIds.length; i++) {
```

```
507         if (orderIds[i] == id) {
508             if (i != orderIds.length - 1) {
509                 orderIds[i] = orderIds[orderIds.length - 1];
510             }
511             orderIds.pop();
512             break;
513         }
514     }
515 }
```

Listing 2.4: XAUMCDA.sol

Impact It may cause DoS in `_claimXaum()` and `_closeOrder()` functions.

Suggestion Limit the maximum number of orders each user can create.

Feedback from the project The project team considers that creating multiple orders will only affect the user himself, and there is a minimum fixed investment amount limit for creating orders, which has capital costs, so it will not be modified here for the time being.

2.1.4 Potential DoS or unfairness due to incorrect calculation of allowanceAmount

Severity Low

Status Confirmed

Introduced by Version 1

Description In the contract `XAUMDCA`, the function `_swapDollarToStableToken()` calculates the `allowanceAmount` with `getAmountByShare(order.dollarShareBalance) + 1`. However, this calculation might cause DoS to other users. For example, we assume there are user1 and user1.

1. They both transfer 150 rebase dollars to the DCA and then the DCA holds 300 rebase dollars.
2. User1's `dollarShareBalance` and user2's `dollarShareBalance` are both 150.
3. User1's `dollarPerTrade` and user2's `dollarPerTrade` are both 50.
4. After a time of interval, the DCA still holds 300 dollars.
5. The operator executes the user1's order, `allowanceAmount` will be 55 and the DCA holds 245 dollars after swap. In this case, the user1's `dollarBalance` will be 95 and `dollarShareBalance` is 95. This case will happen as mentioned in Note 3.
6. The operator executes the user2's order, `allowanceAmount` will be 55 and the DCA holds 190 dollars after swap. In this case, the user2's `dollarBalance` will be 95 and `dollarShareBalance` is 95. This case will happen as mentioned in Note 3.
7. After a time of interval, the DCA holds 200 dollars due rebasing and 190 totalShares.
8. The operator executes the user1's order, `allowanceAmount` will be 101 since `dollarBalance < 2*dollarPerTrade`. And the user1's `dollarBalance` will be 0 and user1's `dollarShareBalance` will be 0 as the case mentioned in DeFi Security 2.
9. After step 8, the DCA holds 99 dollars and 95 total shares.
10. The operator executes the user2's order, `allowanceAmount` will be 100 since `dollarBalance < 2*dollarPerTrade`. However, the DCA only holds 99 dollars which can lead to DoS or

unfairness to the user2.

```
345 function _swapDollarToStableToken(Order storage order, address adapter, bytes memory data)
    internal returns (uint dollarDelta, uint stableTokenDelta){
346     // Cache storage variables
347     uint256 dollarBalance = order.dollarBalance;
348     uint256 dollarPerTrade = order.dollarPerTrade;
349     address dollarAddr = dollar;
350     address stableTokenAddr = stableToken;
351
352     // Get balances once
353     uint256 dollarBalanceBefore = IERC20(dollarAddr).balanceOf(address(this));
354     uint256 stableTokenBalanceBefore = IERC20(stableTokenAddr).balanceOf(address(this));
355
356     // Calculate allowance amount
357     uint256 allowanceAmount;
358     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
359         allowanceAmount = getAmountByShare(order.dollarShareBalance) + 1; // for share amount
360         convert calculate precision
361     } else {
362         allowanceAmount = dollarPerTrade;
363     }
364     IERC20(dollarAddr).forceApprove(adapter, allowanceAmount * 11 / 10); // approve more
365
366     // Execute swap
367     (bool success, bytes memory result) = adapter.call(data);
368     if (!success) {
369         revert CallFailed(result);
370     }
371
372     // Get balances after swap
373     uint256 dollarBalanceAfter = IERC20(dollarAddr).balanceOf(address(this));
374     uint256 stableTokenBalanceAfter = IERC20(stableTokenAddr).balanceOf(address(this));
375
376     // Calculate deltas
377     dollarDelta = dollarBalanceBefore - dollarBalanceAfter;
378     stableTokenDelta = stableTokenBalanceAfter - stableTokenBalanceBefore;
379
380     // Validate deltas
381     require(dollarDelta > 0, 'DCA_DOLLAR_DELTA_MUST_POSITIVE');
382     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
383         require(dollarDelta <= allowanceAmount, 'DCA_INVALID_DOLLAR_DELTA');
384     } else {
385         require(dollarDelta <= dollarPerTrade * 11 / 10 && dollarDelta <= dollarBalance, '
            DCA_INVALID_DOLLAR_DELTA'); // allow trade a little more than dollarPerTrade every
            trade
386     }
387     require(stableTokenDelta > 0, 'DCA_INVALID_STABLE_TOKEN_AMOUNT');
388
389     // Validate price
390     uint price = uint224(uint224(stableTokenDelta) * (2**112) / uint112(dollarDelta));
391     require(price >= minDollarPriceAllowed, 'DCA_INVALID_DOLLAR_PRICE');
```

```
392     return (dollarDelta, stableTokenDelta);
393 }
```

Listing 2.5: XAUMCDA.sol

Impact This calculation might cause DoS or unfairness to other users.

Suggestion Change the calculation of the allowanceAmount to be `getAmountByShare(order.dollarShareBalance) + 1` when `dollarBalance < 2*dollarPerTrade`.

Feedback from the project The operator is the operating account of the project party. It will swap and invest according to the `perTradeAmount`, not according to the number of approvals. They believe that this problem should not occur.

2.2 DeFi Security

2.2.1 The pending XAUM may be lost

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description When the operator role executes the `closeOrderByOperator()` function, it does not check whether the value of `order.xaumPending` is zero. In this case, the XAUM tokens pending in the order cannot be collected.

```
454 function closeOrderByOperator(uint64 id) public onlyOperator {
455     Order storage order = orders[id];
456     require(userBlacklist[order.owner] == true, 'DCA_NOT_IN_BLACKLIST');
457     _closeOrder(order, order.owner, id, legalAccount);
458 }
```

Listing 2.6: XAUMCDA.sol

```
460 function _closeOrder(Order storage order, address user, uint64 id, address receiver) internal
461     {
462         require(order.status == STATUS_ACTIVE, 'DCA_INVALID_ORDER_STATUS');
463
464         uint xaumBalance = order.xaumBalance;
465         uint dollarBalance = order.dollarBalance;
466
467         if (xaumBalance > 0) {
468             order.xaumBalance = 0;
469             IERC20(xaum).safeTransfer(receiver, xaumBalance);
470         }
471         if (dollarBalance > 0) {
472             order.dollarBalance = 0;
473             if (isRebaseToken) {
474                 uint256 shareBalance = order.dollarShareBalance;
475                 order.dollarShareBalance = 0;
476                 dollarBalance = getAmountByShare(shareBalance);
477                 totalShares -= shareBalance;
478             }
479         }
480     }
```

```

477     }
478     IERC20(dollar).safeTransfer(receiver, dollarBalance);
479 }
480 order.status = STATUS_CANCELED;
481 _removeUserOrder(user, id);
482 emit CloseOrder(user, id, receiver, xaumBalance, dollarBalance);
483 }

```

Listing 2.7: XAUMCDA.sol

Impact Users may lose the pending XAUM tokens.

Suggestion Ensure there are no XAUM tokens pending in the order in function `closeOrderByOperator()`.

2.2.2 Incorrect round down in function `_burnShareAfterSendDollar()`

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description For rebasing dollar tokens, there is a potential rounding vulnerability in the calculation of burned shares. Specifically, when user orders are executed, the shares corresponding to the spending dollar are deduced from `dollarShareBalance`. The deduced shares are calculated via the `_burnShareAfterSendDollar()` function, which calculates the result as: `totalShares * dollarAmount / (balance + dollarAmount)`.

Consequently, the result rounds down, allowing users to burn fewer shares than required. The remaining shares are therefore inflated when the order is completed, increasing the refund amount to users. It is important to note that, applying rounding up fixes this vulnerability while introducing a potential DoS risk as follows: Due to the rounding up, the calculation of `dollarShareBalance`, `order.dollarShareBalance -= _burnShareAfterSendDollar(dollarDelta)` may be reverted. The handling logic for this edge case must be implemented during fixes.

```

303 function _executeOrder(uint64 id, address adapter, bytes memory data) internal {
304     Order storage order = orders[id];
305     require(order.status == STATUS_ACTIVE, 'DCA_INVALID_ORDER_STATUS');
306     require(block.timestamp >= order.lastTradeTime + order.interval * 95 / 100, '
        DCA_NOT_TIME_TO_TRADE');
307
308     uint dollarDelta = order.dollarPerTrade;
309     uint stableTokenDelta = order.dollarPerTrade;
310     if (!dollarIsStableToken) {
311         require(adapterWhitelist[adapter] == true, 'DCA_ADAPTER_NOT_IN_WHITELIST');
312         (dollarDelta, stableTokenDelta) = _swapDollarToStableToken(order, adapter, data);
313     }
314     uint256 feeAmount = fee;
315     require(stableTokenDelta > feeAmount, 'DCA_INVALID_STABLE_TOKEN_DELTA');
316     feeToClaim += feeAmount;
317     stableTokenDelta -= feeAmount;
318
319     if (dollarDelta <= order.dollarBalance) {

```

```
320     order.dollarBalance -= dollarDelta;
321 } else {
322     order.dollarBalance = 0;
323 }
324 if (isRebaseToken) {
325     order.dollarShareBalance -= _burnShareAfterSendDollar(dollarDelta);
326 }
327 order.lastTradeTime = uint64(block.timestamp);
328
329 IERC20(stableToken).safeIncreaseAllowance(minter, stableTokenDelta);
330 uint256 amountOut = IXAUMMinter(minter).swapForXAUM(order.owner, stableToken,
    stableTokenDelta);
331 order.xaumPending += amountOut;
332 if (order.dollarBalance == 0) {
333     order.status = STATUS_COMPLETED_WITHOUT_COLLECT;
334     if (order.dollarShareBalance > 0) {
335         uint256 shareBalance = order.dollarShareBalance;
336         order.dollarShareBalance = 0;
337         uint transferAmount = getAmountByShare(shareBalance);
338         totalShares -= shareBalance;
339         IERC20(dollar).safeTransfer(order.receiver, transferAmount);
340     }
341 }
342 emit XaumConvert(order.owner, id, dollarDelta, stableTokenDelta, amountOut, feeAmount);
343 }
```

Listing 2.8: XAUMCDA.sol

```
497 function _burnShareAfterSendDollar(uint256 dollarAmount) internal returns (uint256 share) {
498     uint256 balance = IERC20(dollar).balanceOf(address(this));
499     share = totalShares * dollarAmount / (balance + dollarAmount);
500     totalShares -= share;
501     return share;
502 }
```

Listing 2.9: XAUMCDA.sol

Impact Users could get more refunding dollar tokens from remaining shares when the order is completed.

Suggestion Apply round-up in the burned shares calculation, and handle the edge cases to make sure the calculation `order.dollarShareBalance -= _burnShareAfterSendDollar(dollarDelta)` will not be reverted.

2.2.3 Potential inflation attack in function `createOrder()`

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the [XAUMDCA](#) contract, when the value of `isRebaseToken` is true, the `createOrder()` function will call the `_mintShareAfterReceiveDollar()` function. Due to the lack of checks on

the minted shares, attackers can launch the inflation attack here. Specifically, the attacker can first execute the `createOrder()` function when the totalshares is 0. After that, the attacker can directly transfer a large amount of dollars to the `XAUMCDA` contract, causing the dollar balance in the contract to become very large. In this case, when the other users invoke the `createOrder()` function with a rather small dollar value, the minted shares can be zero. Finally, the attacker can execute the `closeOrder()` function, and then all the dollars in the contract can be taken away, including the victim's dollars.

```
256 function createOrder(address user, uint initDollarAmount, uint amountPerTrade, uint64 interval
    , address receiver) public onlyRouter whenNotPaused {
257     require(amountPerTrade >= minDollarAmount, 'DCA_INVALID_AMOUNT_PER_TRADE');
258     require(initDollarAmount >= amountPerTrade && initDollarAmount % amountPerTrade == 0, '
        DCA_INVALID_INIT_DOLLAR_AMOUNT');
259     require(interval >= minTradeInterval && interval % minTradeInterval == 0, '
        DCA_INVALID_TRADE_INTERVAL');
260     require(userBlacklist[user] == false && userBlacklist[receiver] == false, 'DCA_IN_BLACKLIST
        ');
261
262     Order memory order;
263     order.id = uint64(orders.length);
264     order.owner = user;
265     order.dollarInitBalance = initDollarAmount;
266     order.dollarPerTrade = amountPerTrade;
267     order.interval = interval;
268     order.lastTradeTime = uint64(0);
269     order.dollarBalance = initDollarAmount;
270     order.receiver = receiver;
271     if (isRebaseToken) {
272         order.dollarShareInitAmount = _mintShareAfterReceiveDollar(initDollarAmount);
273         order.dollarShareBalance = order.dollarShareInitAmount;
274     }
275     order.status = STATUS_ACTIVE;
276     orders.push(order);
277     userOrders[user].push(order.id);
278     emit NewOrder(user, order.id, initDollarAmount, amountPerTrade, interval, receiver);
279 }
```

Listing 2.10: XAUMCDA.sol

```
485 function _mintShareAfterReceiveDollar(uint256 dollarAmount) internal returns (uint256 share) {
486     uint256 balance = IERC20(dollar).balanceOf(address(this));
487     if (totalShares == 0) {
488         totalShares = dollarAmount;
489         return dollarAmount;
490     } else {
491         share = totalShares * dollarAmount / (balance - dollarAmount);
492         totalShares += share;
493         return share;
494     }
495 }
```

Listing 2.11: XAUMCDA.sol

Impact The dollars that users use to create orders may be taken away by attackers.

Suggestion Make sure the minted share is greater than zero.

2.2.4 Lack of revoking approvals

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `_swapDollarToStableToken()` function, the contract calls the `forceApprove()` function to authorize more dollars for the adapter. There is a risk that the adapter can manipulate the `XAUMDCA` contract assets.

```
345 function _swapDollarToStableToken(Order storage order, address adapter, bytes memory data)
    internal returns (uint dollarDelta, uint stableTokenDelta){
346     // Cache storage variables
347     uint256 dollarBalance = order.dollarBalance;
348     uint256 dollarPerTrade = order.dollarPerTrade;
349     address dollarAddr = dollar;
350     address stableTokenAddr = stableToken;
351
352     // Get balances once
353     uint256 dollarBalanceBefore = IERC20(dollarAddr).balanceOf(address(this));
354     uint256 stableTokenBalanceBefore = IERC20(stableTokenAddr).balanceOf(address(this));
355
356     // Calculate allowance amount
357     uint256 allowanceAmount;
358     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
359         allowanceAmount = getAmountByShare(order.dollarShareBalance) + 1; // for share amount
360         // convert calculate precision
361     } else {
362         allowanceAmount = dollarPerTrade;
363     }
364     IERC20(dollarAddr).forceApprove(adapter, allowanceAmount * 11 / 10); // approve more
365
366     // Execute swap
367     (bool success, bytes memory result) = adapter.call(data);
368     if (!success) {
369         revert CallFailed(result);
370     }
371
372     // Get balances after swap
373     uint256 dollarBalanceAfter = IERC20(dollarAddr).balanceOf(address(this));
374     uint256 stableTokenBalanceAfter = IERC20(stableTokenAddr).balanceOf(address(this));
375
376     // Calculate deltas
377     dollarDelta = dollarBalanceBefore - dollarBalanceAfter;
378     stableTokenDelta = stableTokenBalanceAfter - stableTokenBalanceBefore;
379
380     // Validate deltas
381     require(dollarDelta > 0, 'DCA_DOLLAR_DELTA_MUST_POSITIVE');
382     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
```

```
382     require(dollarDelta <= allowanceAmount, 'DCA_INVALID_DOLLAR_DELTA');
383 } else {
384     require(dollarDelta <= dollarPerTrade * 11 / 10 && dollarDelta <= dollarBalance, '
        DCA_INVALID_DOLLAR_DELTA'); // allow trade a little more than dollarPerTrade every
        trade
385 }
386 require(stableTokenDelta > 0, 'DCA_INVALID_STABLE_TOKEN_AMOUNT');
387
388 // Validate price
389 uint price = uint224(uint224(stableTokenDelta) * (2**112) / uint112(dollarDelta));
390 require(price >= minDollarPriceAllowed, 'DCA_INVALID_DOLLAR_PRICE');
391
392 return (dollarDelta, stableTokenDelta);
393 }
```

Listing 2.12: XAUMCDA.sol

Impact The adapter can directly operate the dollar amount authorized by the [XAUMDCA](#) contract.

Suggestion After executing the swap operation, the number of allowance should be set to be zero.

2.2.5 Lack of check on `minDollarPriceAllowed` variable

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `_swapDollarToStableToken()` function, the `minDollarPriceAllowed` variable is used as a slippage check. However, this value may be zero and there is a possibility of a sandwich attack.

```
345 function _swapDollarToStableToken(Order storage order, address adapter, bytes memory data)
    internal returns (uint dollarDelta, uint stableTokenDelta){
346     // Cache storage variables
347     uint256 dollarBalance = order.dollarBalance;
348     uint256 dollarPerTrade = order.dollarPerTrade;
349     address dollarAddr = dollar;
350     address stableTokenAddr = stableToken;
351
352     // Get balances once
353     uint256 dollarBalanceBefore = IERC20(dollarAddr).balanceOf(address(this));
354     uint256 stableTokenBalanceBefore = IERC20(stableTokenAddr).balanceOf(address(this));
355
356     // Calculate allowance amount
357     uint256 allowanceAmount;
358     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
359         allowanceAmount = getAmountByShare(order.dollarShareBalance) + 1; // for share amount
        convert calculate precision
360     } else {
361         allowanceAmount = dollarPerTrade;
```

```
362     }
363     IERC20(dollarAddr).forceApprove(adapter, allowanceAmount * 11 / 10); // approve more
364
365     // Execute swap
366     (bool success, bytes memory result) = adapter.call(data);
367     if (!success) {
368         revert CallFailed(result);
369     }
370
371     // Get balances after swap
372     uint256 dollarBalanceAfter = IERC20(dollarAddr).balanceOf(address(this));
373     uint256 stableTokenBalanceAfter = IERC20(stableTokenAddr).balanceOf(address(this));
374
375     // Calculate deltas
376     dollarDelta = dollarBalanceBefore - dollarBalanceAfter;
377     stableTokenDelta = stableTokenBalanceAfter - stableTokenBalanceBefore;
378
379     // Validate deltas
380     require(dollarDelta > 0, 'DCA_DOLLAR_DELTA_MUST_POSITIVE');
381     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
382         require(dollarDelta <= allowanceAmount, 'DCA_INVALID_DOLLAR_DELTA');
383     } else {
384         require(dollarDelta <= dollarPerTrade * 11 / 10 && dollarDelta <= dollarBalance, '
            DCA_INVALID_DOLLAR_DELTA'); // allow trade a little more than dollarPerTrade every
            trade
385     }
386     require(stableTokenDelta > 0, 'DCA_INVALID_STABLE_TOKEN_AMOUNT');
387
388     // Validate price
389     uint price = uint224(uint224(stableTokenDelta) * (2**112) / uint112(dollarDelta));
390     require(price >= minDollarPriceAllowed, 'DCA_INVALID_DOLLAR_PRICE');
391
392     return (dollarDelta, stableTokenDelta);
393 }
```

Listing 2.13: XAUMCDA.sol

Impact If `minDollarPriceAllowed` is not set, it is possible to suffer from sandwich attacks.

Suggestion In the `_swapDollarToStableToken()` function, check if the value of `minDollarPriceAllowed` is set properly.

2.2.6 Potential overflow during type casting

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The data type of `dollarDelta` is `uint256`. Casting it explicitly to `uint112` types may result in an overflow error. The same issue also exists in the casting of `stableTokenDelta` from `uint256` types to `uint224`.

```
345 function _swapDollarToStableToken(Order storage order, address adapter, bytes memory data)
    internal returns (uint dollarDelta, uint stableTokenDelta){
346     // Cache storage variables
347     uint256 dollarBalance = order.dollarBalance;
348     uint256 dollarPerTrade = order.dollarPerTrade;
349     address dollarAddr = dollar;
350     address stableTokenAddr = stableToken;
351
352     // Get balances once
353     uint256 dollarBalanceBefore = IERC20(dollarAddr).balanceOf(address(this));
354     uint256 stableTokenBalanceBefore = IERC20(stableTokenAddr).balanceOf(address(this));
355
356     // Calculate allowance amount
357     uint256 allowanceAmount;
358     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
359         allowanceAmount = getAmountByShare(order.dollarShareBalance) + 1; // for share amount
            convert calculate precision
360     } else {
361         allowanceAmount = dollarPerTrade;
362     }
363     IERC20(dollarAddr).forceApprove(adapter, allowanceAmount * 11 / 10); // approve more
364
365     // Execute swap
366     (bool success, bytes memory result) = adapter.call(data);
367     if (!success) {
368         revert CallFailed(result);
369     }
370
371     // Get balances after swap
372     uint256 dollarBalanceAfter = IERC20(dollarAddr).balanceOf(address(this));
373     uint256 stableTokenBalanceAfter = IERC20(stableTokenAddr).balanceOf(address(this));
374
375     // Calculate deltas
376     dollarDelta = dollarBalanceBefore - dollarBalanceAfter;
377     stableTokenDelta = stableTokenBalanceAfter - stableTokenBalanceBefore;
378
379     // Validate deltas
380     require(dollarDelta > 0, 'DCA_DOLLAR_DELTA_MUST_POSITIVE');
381     if (isRebaseToken && dollarBalance < 2 * dollarPerTrade) {
382         require(dollarDelta <= allowanceAmount, 'DCA_INVALID_DOLLAR_DELTA');
383     } else {
384         require(dollarDelta <= dollarPerTrade * 11 / 10 && dollarDelta <= dollarBalance, '
            DCA_INVALID_DOLLAR_DELTA'); // allow trade a little more than dollarPerTrade every
            trade
385     }
386     require(stableTokenDelta > 0, 'DCA_INVALID_STABLE_TOKEN_AMOUNT');
387
388     // Validate price
389     uint price = uint224(uint224(stableTokenDelta) * (2**112) / uint112(dollarDelta));
390     require(price >= minDollarPriceAllowed, 'DCA_INVALID_DOLLAR_PRICE');
391
392     return (dollarDelta, stableTokenDelta);
```

```
393 }
```

Listing 2.14: XAUMCDA.sol

Impact The calculated result can be wrong.

Suggestion Add a maximum value check of `dollarPerTrade` in the function `createOrder()`.

2.2.7 Lack of sanity checks in function `setDCA()`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description Each DCA is dedicated to one dollar token and vice versa, according to the current implementation. However, the `setDCA()` function lacks a sanity check on the dollar parameter against the DCA contracts's dollar variable.

```
18 function setDCA(address dollar, address dca) onlyOwner public {
19     dcaSet[dollar] = dca;
20 }
```

Listing 2.15: XAUMDCARouter.sol

Impact Misconfigurations on the dollar-dca mapping could disrupt the Router's functionality.

Suggestion Apply sanity checks on the dollar against `IXAUMDCA(dca).dollar()`.

2.2.8 Improper order completion logic

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description The `_executeOrder()` function sets an order status as completed when `order.dollarBalance` reaches zero (line 305). However, the function fails to handle the case where `dollarShareBalance` becomes zero while `dollarBalance` remains nonzero. At this time, the order should be deemed completed as well. This scenario could happen in some extreme cases, such as a deflation in token balances due to negative interests.

```
303 function _executeOrder(uint64 id, address adapter, bytes memory data) internal {
304     Order storage order = orders[id];
305     require(order.status == STATUS_ACTIVE, 'DCA_INVALID_ORDER_STATUS');
306     require(block.timestamp >= order.lastTradeTime + order.interval * 95 / 100, '
        DCA_NOT_TIME_TO_TRADE');
307     //...
308     if (order.dollarBalance == 0) {
309         order.status = STATUS_COMPLETED_WITHOUT_COLLECT;
310         if (order.dollarShareBalance > 0) {
311             uint256 shareBalance = order.dollarShareBalance;
312             order.dollarShareBalance = 0;
313             uint transferAmount = getAmountByShare(shareBalance);
```

```

314         totalShares -= shareBalance;
315         IERC20(dollar).safeTransfer(order.receiver, transferAmount);
316     }
317 }
318 emit XaumConvert(order.owner, id, dollarDelta, stableTokenDelta, amountOut, feeAmount);
319 }

```

Listing 2.16: XAUMDCA.sol

Impact The order cannot be completed as intended, and users have to manually close orders to collect the refundments.

Suggestion Implement the code logic to handle the edge case.

Feedback from the project The project does not support deflation rebase tokens, so this situation should not exist.

2.2.9 The fixedPrice for all the stable tokens is the same

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the [XAUMDCAMinter](#) contract, there is [usdWhitelist](#), which means there might be more than one stableTokens. Meanwhile, the fixedPrice for all the stable tokens is the same. However, in reality, the exchange rate of [USDC](#) and [USDT](#) may not be strictly 1:1, which might provide arbitrage opportunities.

```

303 function _executeOrder(uint64 id, address adapter, bytes memory data) internal {
304     Order storage order = orders[id];
305     require(order.status == STATUS_ACTIVE, 'DCA_INVALID_ORDER_STATUS');
306     require(block.timestamp >= order.lastTradeTime + order.interval * 95 / 100, '
307         DCA_NOT_TIME_TO_TRADE');
308     //...
309     if (order.dollarBalance == 0) {
310         order.status = STATUS_COMPLETED_WITHOUT_COLLECT;
311         if (order.dollarShareBalance > 0) {
312             uint256 shareBalance = order.dollarShareBalance;
313             order.dollarShareBalance = 0;
314             uint transferAmount = getAmountByShare(shareBalance);
315             totalShares -= shareBalance;
316             IERC20(dollar).safeTransfer(order.receiver, transferAmount);
317         }
318     }
319     emit XaumConvert(order.owner, id, dollarDelta, stableTokenDelta, amountOut, feeAmount);
320 }

```

Listing 2.17: XAUMDCA.sol

Impact When the exchange rate of [USDC](#) and [USDT](#) may not be strictly 1:1, which might provide arbitrage opportunities.

Suggestion Set different prices for different stable tokens.

2.3 Additional Recommendation

2.3.1 Lack of invoking function `_disableInitializers()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The [XAUMDCA](#) and [XAUMDCAMinter](#) contracts are both upgradeable. In their constructors, the `_disableInitializers()` function should be invoked to prevent unexpected initialization on the implementation contract.

```
43  contract XAUMDCA is OwnableUpgradeable, PausableUpgradeable, DelayedUpgradeable, IXAUMDCA {
```

Listing 2.18: XAUMDCA.sol

```
34  contract XAUMDCAMinter is DelayedUpgradeable, IXAUMMinter {
```

Listing 2.19: XAUMDCAMinter.sol

Suggestion Invoke the `_disableInitializers()` function in the constructors.

2.3.2 Redundant rounding up in total fee calculation

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The `getTotalFee()` function contains a redundant round-up operation. As the `initAmount` is enforced to be an integer multiple of `amountPerTrade` upon order creation, the formula in this function can be simplified as $(\text{fee} * \text{initAmount} / \text{amountPerTrade})$. It is important to note that the number of order executions can be affected by the potential inflation or deflation in dollar assets, causing the final fee to be different from estimates.

```
555  function getTotalFee(uint initAmount, uint amountPerTrade) public view returns (uint256) {
556      return fee * ((initAmount + amountPerTrade - 1) / amountPerTrade);
557  }
```

Listing 2.20: XAUMDCA.sol

Suggestion Remove the redundant round-up operation.

Feedback from the project After internal communication, the project team felt that this interface was not needed and deleted it.

2.3.3 Redundant [XAUMDCAForTest](#)

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The contract [XAUMDCA](#) defines the [XAUMDCAForTest](#) which is redundant test code.

```
560     contract XAUMDCAFForTest is XAUMDCA {
561         function setRebaseTokenToggle(bool _isRebaseToken) public {
562             isRebaseToken = _isRebaseToken;
563         }
564     }
```

Listing 2.21: XAUMDCA.sol

Suggestion Remove the redundant test code.

Feedback from the project This contract was only used in the unit tests (UT). Based on the feedback, the contract has been deleted, and the corresponding tests have also been modified.

2.4 Note

2.4.1 Ensure enough XAUM tokens to pay users in XAUMDCAMinter contract

Introduced by [Version 1](#)

Description In the [XAUMDCAMinter](#) contract, XAUM tokens are directly transferred by the project party. The amount of transfer depends on the project party, but it must be ensured that the number of XAUM tokens in the [XAUMDCAMinter](#) contract is sufficient to pay for the user's collection.

2.4.2 Potential centralization risk

Introduced by [Version 1](#)

Description Several protocol roles could conduct privileged operations, which introduces potential centralization risks. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.4.3 Excess allowance can reduce transaction count

Introduced by [Version 1](#)

Description In the XAUMDCA contract, the allowance for adapter will be approved with $\text{allowanceAmount} * 11 / 10$. In that way, the trade times might be less than expected. For example, when the $\text{dollarBalance}=110$ and $\text{dollarPerTrade}=10$, it will take 11 times to finish the order normally. However, with the carefully constructed input param data, it only needs 10 times to finish the order due to the $\text{allowanceAmount} * 11 / 10$.

Feedback from the Project This is to take into account the problem of calculation accuracy when converting shares to amuons. Sometimes the calculated amount will be a little less. For convenience, we approve about 10% more. However, when the operator actually executes the order, it will strictly follow the pertrade quantity for fixed investment. Only when the operator makes the last fixed investment of each order, it will calculate the amount corresponding to the share (this part also takes the interest into consideration), and invest the remaining principal and all interest.

2.4.4 Not checking the `data` parameter of the `_executeOrder()` function

Introduced by [Version 1](#)

Description In the [XAUMDCA](#) contract, the input param `data` of the function `_executeOrder()` is unreliable and needs to be checked.

Feedback from the Project The `data` parameter is fed to the adapter contract in the whitelist. What it can do should be limited. And we don't have a clear check item here, because it may require different decoding and checking for different adapters.

